# Alto Engine 設計書

Oct 18, 2026 · @mimi

## 概要

Alto Engineは、ドキュメント・履歴・GPU描画・資源管理をすべて1本の「Engine Worker」内のRust/WASMに置き、メインスレッドは薄いTypeScriptプロキシにする。初期の成功条件は「Rust製エンジンがWorker内でWebGPU描画し、ブラシ編集・Undo・保存・復元が通る」ことで、これをマイルストーンM4（MVP）とする。

設計の芯は一つ。**ラスター画素の正本は、CPU側の内容アドレス化タイル（256×256、BLAKE3ハッシュ）に置き、GPUは常に作り直せるキャッシュとして扱う。** これでUndo、保存、版管理、GPUデバイス喪失からの復旧が同じ仕組みに乗る。

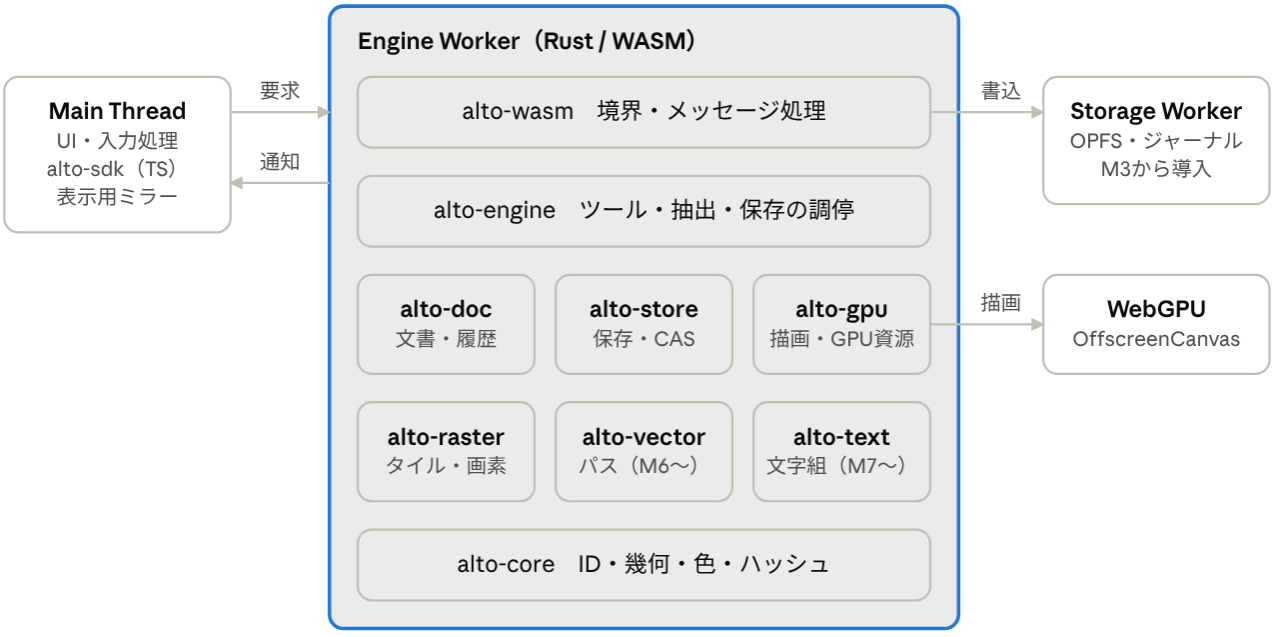
| 論点 | 推奨 | 主な理由 |
|---|---|---|
| 描画バックエンド | Rustの`wgpu`をEngine Worker内で使い、OffscreenCanvasへ描画 | 描画コードと文書が同じメモリ空間にあり、JS⇔WASM往復が増えない。ネイティブへそのまま移植できる |
| ラスターの正本 | CPU側の内容アドレス化タイル | GPU結果をハッシュで確定させるので、GPUごとの演算差が履歴に混ざらない |
| 履歴 | 可逆Op＋タイルハッシュの差し替え | ラスターのUndoが画素コピーなしで済む |
| Render Graph | 汎用グラフは作らず、Scene→RenderPlanの軽量コンパイル | 分離が必要なノードだけをオフスクリーン化する。パスは4種類で足りる |
| スレッド | MVPはMain＋Engine Workerの2本。Storage/Compute Workerは後から追加 | SharedArrayBufferなしで動くことを基本線にする |
| ファイル形式 | manifest＋ハッシュ名オブジェクト群。単一ファイル（ZIP格納）とフォルダの2表現 | フォルダ形式がそのままGitに載る |
| ECS・CRDT | 採用しない | 現要件では必要性を説明できない。IDとOpの設計で将来の余地だけ残す |

今回の対象外（エディタUI、Drive連携の実装、共同編集、AI機能など）は指示書の第15節のとおり。ただしストレージと版管理は、それらを後から差し込める境界を先に切っておく。

# 第1部　アーキテクチャ

Engine Worker内のRustが状態の正本をすべて持ち、TypeScriptは要求を送って通知を受け取るだけにする。正本が1か所にしかないので、同期のずれや二重管理が構造的に起きない。

**重い処理はすべてEngine Worker内のRustで完結する**




全体構成 · 3スレッド、9 crate

Engine Worker内では下の段ほど基盤で、依存は常に上から下へ一方向。Mainとの間は要求と通知の非同期メッセージだけでつながる。

## 1.1 crate構成

当初は9 crateで始める。指示書の11コンポーネントとの対応は下表のとおり。循環依存か再利用の理由がない分割はしない。

| crate | 含むコンポーネント | 責任 | 依存先 | 導入 |
|---|---|---|---|---|
| `alto-core` | Core | ID、幾何（`Affine2`・`Rect`）、色、画素形式、ハッシュ、共通エラー | なし | M1 |
| `alto-raster` | Raster（CPU側） | タイル格子、CPU画素演算、ブラシのダブ生成（形状・補間・筆圧） | core | M1 |
| `alto-doc` | Document、History | Scene Graph、Op、Transaction、Undo/Redo、スナップショット、意味的差分、直列化 | core | M1 |

| crate | 含むコンポーネント | 責任 | 依存先 | 導入 |
|---|---|---|---|---|
| `alto-store` | Storage Abstraction | 内容アドレスストア（CAS）、パッケージ形式、ジャーナル、保存先trait | core | M3 |
| `alto-gpu` | Graphics、Resource Manager、Effects（GPU） | wgpu、RenderScene/RenderPlan、合成、タイルキャッシュ、テクスチャ管理、WGSL | core、raster | M0 |
| `alto-vector` | Vector | パス表現、テッセレーション→メッシュ | core | M6 |
| `alto-text` | Text | シェーピング、組版（縦書き含む）、グリフ→パス | core、vector | M7 |
| `alto-engine` | 統合 | ツールセッション、文書→RenderSceneの抽出、保存の調停、Rust向け公開API | 上記すべて | M1 |
| `alto-wasm` | WASM Runtime | wasm-bindgen境界、Worker内ループ、メッセージプロトコル、Surface生成 | engine | M0 |

意図して分けなかったもの:

- **History**は`alto-doc`内のモジュールにする。Opと文書状態は密結合で、別crateにしても依存の矢印が増えるだけ。Git Adapterだけは後で`alto-git`として外に出す。
- **Effects**は、パラメータ定義を`alto-doc`、GPU実装を`alto-gpu`、CPU実装を`alto-raster`に置く。エフェクトが10種を超えたら`alto-effects`へ切り出す。
- **Resource Manager**は`alto-gpu`内に置く。GPUハンドルを触る唯一の場所なので同居が自然。

## 1.2 依存ルール

1. `alto-core`は他のどのcrateにも依存しない。
2. `alto-doc`はGPUを知らず、`alto-gpu`は文書を知らない。両者をつなぐのは`alto-engine`の「シーン抽出」だけ。
3. `alto-gpu`が受け取るのは描画専用の中間表現`RenderScene`（読み取り専用）。「GraphicsはDocumentを直接変更しない」が型で保証される。
4. ファイル・IndexedDB・OPFSへのIOは`alto-store`のtraitの向こう側。Rust側はバイト列とハッシュしか扱わない。

5. `web-sys`を直接使ってよいのは`alto-wasm`だけ。OffscreenCanvasからの`wgpu::Surface`生成もここで行い、`alto-gpu`へ渡す。結果として`alto-wasm`以外はネイティブでビルド・テストできる。

## 1.3 RustとTypeScriptの責任分担

| 領域 | Rust（Engine Worker） | TypeScript |
|---|---|---|
| 文書状態 | 正本を保持し、変更する | 読み取り専用のミラーをパッチで更新 |
| 描画 | wgpuで合成から提示まで全部 | canvasを`transferControlToOffscreen()`で渡すだけ |
| 入力 | ツールセッションが解釈してOpにする | PointerEventを集約し、フレーム単位で転送 |
| 画像の取り込み | 画素を受け取りタイル化・ハッシュ化 | `createImageBitmap`でデコード |
| 保存 | パッケージのバイト列とハッシュを生成 | `StorageProvider`が実際に読み書き |
| エラー | 構造化エラーを返す | `AltoError`に変換して公開 |

## 1.4 データフロー（1回の編集）

1. Main：PointerEventを`getCoalescedEvents()`で集め、1フレーム分を`Float32Array`にまとめてWorkerへ転送（コピーなしのtransfer）。
2. Engine：ツールセッション（例：ブラシ）が入力をOpに変換し、Transactionを開く。
3. Document：Opを適用し、`ChangeSet`（変更ノード・変更領域・変更タイル）を返す。
4. ChangeSetを4か所へ配る。
   - 履歴：Transactionのコミット時にUndoスタックへ積む
   - 描画：変更ノードだけ`RenderScene`を差し替え、変更領域の表示タイルを無効化
   - ジャーナル：コミット済みTransactionをOPFSへ追記（M3から）
   - Main：文書パッチと再描画通知をイベントで送る
5. 次のフレームで、無効化された表示タイルだけを再合成して提示する。

フレームはMainの`requestAnimationFrame`から送る`tick`で駆動する。Worker内のrAFは、対応を確認できたブラウザでのみ使う最適化として扱う。

# 第2部　内部データ構造

文書は「IDで参照しあう木」と「ハッシュで参照する画素」だけでできている。文書内にRustの参照やGPUハンドルを一切持たないので、cloneが安く、Undo・直列化・差分がすべて単純になる。

## 2.1 ID体系

| ID | 形式 | 寿命 | 永続化 | 用途 |
|---|---|---|---|---|
| `NodeId` | u128（UUID v7相当） | 文書と同じ | する | ノードの参照。時刻順に並ぶので差分表示が安定する |
| `AssetId` | u128（UUID v7相当） | 文書と同じ | する | 画像・フォント・ブラシ先端などの論理的な同一性 |
| `ContentHash` | BLAKE3 256bit | 内容と同じ | する | タイル・アセット本体の実体。重複排除と破損検出 |
| `TextureHandle` | index u32＋generation u32 | 実行時のみ | しない | GPUテクスチャの参照。端末ごとに独立 |

NodeIdを連番にしないのは、コピー＆ペースト・文書のマージ・将来の共同編集でIDが衝突しないため。AssetIdと内容ハッシュを分けるのは、リンク切れ画像の差し替えなど「同じアセットのまま中身が変わる」操作を表せるようにするため。

```
// alto-core
#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Serialize,
Deserialize)]
pub struct NodeId(pub u128);
#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Serialize,
Deserialize)]
pub struct AssetId(pub u128);
#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Serialize,
Deserialize)]
pub struct ContentHash(pub [u8; 32]); // BLAKE3

/// 文書座標の変換は f64（巨大キャンバスでの累積誤差対策）。GPUへ渡す時に f32 へ落とす
#[derive(Clone, Copy, PartialEq, Serialize, Deserialize)]
pub struct Affine2(pub [f64; 6]); // [a, b, c, d, e, f]
pub struct Rect { pub x0: f64, pub y0: f64, pub x1: f64, pub y1: f64 }
pub struct IRect { pub x0: i32, pub y0: i32, pub x1: i32, pub y1: i32 }
```

## 2.2 Document と Scene Graph

```
// alto-doc
pub struct Document {
    pub meta: DocumentMeta,
    nodes: HashMap<NodeId, Node>,
    root: NodeId,                              // 常に Group
    pub assets: BTreeMap<AssetId, AssetRecord>,
    pub selection: Option<TileMap>,            // R8 の選択範囲。Undo対象
    revision: u64,                             // 変更ごとに +1。Main側ミラーとの照合
用
}

pub struct DocumentMeta {
    pub width: u32, pub height: u32,
    pub color: ColorSpace,                     // MVP は Srgb のみ
    pub depth: BitDepth,                       // MVP は U8 のみ
    pub format_version: u32,
}
pub enum ColorSpace { Srgb, DisplayP3 }
pub enum BitDepth { U8, U16, F16 }

pub struct Node {
    pub id: NodeId,
    pub parent: Option<NodeId>,
    pub name: String,
    pub props: LayerProps,
    pub kind: NodeKind,
}

pub struct LayerProps {
    pub visible: bool,
    pub locked: bool,
    pub opacity: f32,
    pub blend: BlendMode,
    pub transform: Affine2,                    // 親座標系への変換（非破壊）
    pub mask: Option<Mask>,
    pub clip_to_below: bool,                   // クリッピングマスク
    pub effects: Vec<Effect>,                  // 非破壊エフェクト。先頭から順に適用
}

pub enum NodeKind {
    Group(GroupNode),
    Raster(RasterNode),
    Vector(VectorNode),
    Text(TextNode),
```

```
    Adjustment(AdjustmentNode),
    Unsupported(UnknownNode),               // 新しい形式の未知ノード。保存時にそのまま書き戻す
}

pub struct GroupNode { pub children: Vec<NodeId>, pub isolation: Isolation }
// 下→上の順
pub enum Isolation { PassThrough, Isolated }
pub struct RasterNode { pub tiles: TileMap }
pub struct VectorNode { pub paths: Vec<StyledPath> }
pub struct TextNode { pub content: TextContent, pub frame: TextFrame, pub writing: WritingMode }
pub struct AdjustmentNode { pub adjustment: Adjustment }

pub enum Mask {
    Raster { tiles: TileMap, enabled: bool, linked: bool },   // R8 タイル
    Vector { path: PathData, feather: f32, enabled: bool },
}

pub enum BlendMode {
    Normal, Multiply, Screen, Overlay, Darken, Lighten, ColorDodge, ColorBurn,
    HardLight, SoftLight, Difference, Exclusion, Hue, Saturation, Color, Luminosity, Add,
}
```

- **木の持ち方**：`HashMap<NodeId, Node>`＋親ポインタ＋子の`Vec`。ノード数は数百〜数千で、ECSの利点（大量エンティティの一括処理）が効かないので採用しない。
- **参照**：ノード間・ノード→アセットの参照はすべてID。存在しないIDへの参照はOp適用時に検証で弾く。
- **マスクはノードではなくプロパティ**：レイヤーと一緒に動き、一緒に消えるため。

## 2.3 ラスター：タイルとブラシ

```
// alto-core / alto-raster
pub const TILE: u32 = 256;

#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
pub struct TileKey { pub tx: i32, pub ty: i32 }   // 負の座標も可（キャンバス外へはみ出すレイヤー）

pub struct TileMap {
    pub format: PixelFormat,                        // Rgba8 / R8（マスク・選択範囲）
    pub tiles: BTreeMap<TileKey, ContentHash>,      // 完全透明タイルは持たない
}

/// 画素本体。文書はハッシュしか持たず、本体は TileStore が保持する
pub struct TileData { pub format: PixelFormat, pub pixels: Box<[u8]> } // 256×256、straight alpha

pub trait TileStore {
    fn get(&self, h: &ContentHash) -> Option<Arc<TileData>>;   // メモリ→OPFSキャッシュ→保存先の順に探す
    fn put(&mut self, data: TileData) -> ContentHash;          // 非圧縮画素のハッシュを返す
}
```

保存用の画素は**straight alpha の 8bit**にする。premultipliedで8bitに丸めると低アルファ部の色が失われるため。GPUへ上げる時にシェーダーでpremultiplyする。

```
// ブラシ：将来の筆圧・補間・形状・テクスチャに耐える構造
pub struct BrushTip {
    pub shape: TipShape,
    pub size: f32, pub hardness: f32, pub spacing: f32,   // spacing は直径に対する比率
    pub angle: f32, pub roundness: f32,
    pub texture: Option<AssetId>,                          // テクスチャブラシ
}
pub enum TipShape { Round, Sampled(AssetId) }
pub struct BrushDynamics { pub size: Curve, pub opacity: Curve, pub flow: Curve } // 筆圧・傾き・速度 → 係数
pub struct StrokeSample { pub pos: [f32; 2], pub pressure: f32, pub tilt: [f32; 2], pub t_ms: f64 }
pub struct Dab { pub center: [f32; 2], pub radius: f32, pub opacity: f32, pub angle: f32 }
// 入力サンプル → 補間（Catmull-Rom）→ spacing 間隔の Dab 列 → GPU でまとめてスタンプ
```

ダブ列の生成はCPU（`alto-raster`）で行い、GPUとCPUの両方の描画器が同じダブ列を受け取る。CPU描画器はネイティブのテストと検証の基準に使う。

## 2.4 Asset と Resource Manager

```
// alto-doc（永続）
pub struct AssetRecord {
    pub id: AssetId,
    pub hash: ContentHash,
    pub kind: AssetKind,
    pub byte_len: u64,
    pub dims: Option<(u32, u32)>,
}
pub enum AssetKind { EncodedImage { mime: String }, Font, BrushTip, Pattern }

// alto-gpu（実行時のみ。どこにも永続化しない）
#[derive(Clone, Copy, PartialEq, Eq, Hash)]
pub struct TextureHandle { index: u32, generation: u32 }

/// テクスチャを「作り直せる出どころ」。GPU上にしか無いデータを原則として作らない
#[derive(Clone, PartialEq, Eq, Hash)]
pub enum GpuSource {
    Tile(ContentHash),        // コンテンツタイル
    Asset(AssetId),           // ブラシ先端・パターン・グリフアトラスなど
    Derived(CacheKey),        // 合成結果・エフェクト出力。失っても再計算できる
}

struct Slot { generation: u32, loc: Option<TexLoc>, source: GpuSource, bytes:
u64, last_used: u64, pins: u32 }
enum TexLoc { ArrayLayer { page: u16, layer: u16 }, Standalone(wgpu::Texture)
}

pub struct ResourceManager {
    slots: Vec<Slot>,
    free: Vec<u32>,
    by_source: HashMap<GpuSource, TextureHandle>,
    pages: Vec<TilePage>,     // 256×256 × 256層の texture_2d_array
    budget_bytes: u64,        // 既定 512 MiB。設定で変更可
    used_bytes: u64,
    frame: u64,
}

impl ResourceManager {
    pub fn acquire(&mut self, src: GpuSource) -> Acquire;            //
Ready(handle) か Pending
    pub fn resolve(&self, h: TextureHandle) -> Option<TexRef<'_>>;  // 世代が
```

```
古ければ None
    pub fn pin(&mut self, h: TextureHandle);                          // 描画中
フレームの資源は追い出さない
    pub fn end_frame(&mut self);                                      // pin解
除・予算超過分の追い出し
    pub fn on_device_lost(&mut self);                                 // 全実体
を捨て、全スロットの世代を +1
}
```

- **追い出し順**：pinされていないものを最終使用フレームの古い順（LRU）。同じ古さなら `Derived`（再計算できる）→ `Tile` / `Asset`（CPUから再転送できる）の順。
- **遅延読み込み**：`acquire` が `Pending` を返したら、そのフレームは粗い解像度のタイルで代用し、転送完了後に描き直す。
- **デバイス喪失**：`device.lost` を受けたらデバイスを作り直し、`on_device_lost` で全ハンドルを無効化する。以後はCPU側の正本から必要な分だけ遅延再構築する。進行中のストロークはTransactionごとロールバックする。
- **永続化の禁止**：`TextureHandle` はIndexedDBにもファイルにも書かない。永続するのは `AssetId` と `ContentHash` だけ。
- **コンテンツタイルはテクスチャ配列に格納**：1タイル1テクスチャにするとbind groupの切り替えが増える。配列のページにまとめれば、1回の描画で多数のタイルを扱える。読み込んだ画像も取り込み時にラスタータイルへ変換するので、GPU上の大きな単体テクスチャはほぼ生まれない。

## 2.5 RenderScene と RenderPlan

```
// alto-gpu：文書から抽出した描画専用の中間表現（読み取り専用）
pub struct RenderScene { pub canvas: IRect, pub root: RIdx, pub nodes:
Vec<RNode> }
pub struct RNode {
    pub key: NodeId,                 // キャッシュキー
    pub version: u64,                // 内容が変わるたびに変わる
    pub transform: [f32; 6], pub opacity: f32, pub blend: BlendMode,
    pub mask: Option<RMask>, pub clip_to_below: bool, pub effects:
Vec<REffect>,
    pub content: RContent,
}
pub enum RContent {
    Group { children: Vec<RIdx>, isolated: bool },
    Tiles(Arc<TileMapView>),         // TileKey → GpuSource::Tile
    Mesh(Arc<VectorMesh>),
    Glyphs(Arc<GlyphRun>),
    Adjustment(RAdjustment),
}
```

```
// 1フレーム分の描画計画。Scene とビューポートからコンパイルする
pub struct RenderPlan { pub passes: Vec<Pass>, pub temps: Vec<TargetDesc> }
pub enum Pass {
    /// ハードウェアブレンドで表せる合成。追加メモリなしでターゲットへ直接描く
    Draw { node: RIdx, region: IRect, into: TargetId },
    /// dst を読む必要がある合成（Multiply・Overlay など）。WebGPU には framebuffer
fetch がないためピンポン
    BlendIsolated { src: TargetId, dst: TargetId, out: TargetId, mode:
BlendMode, opacity: f32, mask: Option<TextureHandle> },
    Effect { effect: REffectIdx, input: TargetId, out: TargetId, roi: IRect
},
    Present { from: TargetId, view: ViewTransform },
}
```

汎用のRender Graph（自動リソースエイリアシング・任意のパス依存）は作らない。パスは4種類で、依存関係はシーンの木から自明に決まる。一時ターゲットは（サイズ,形式）をキーにしたプールから借りて、フレーム末に返す。パス数や一時メモリが問題になった時点で、この`RenderPlan`をグラフに昇格させる。

## 2.6 Command・Op・Transaction

**Command**は上位の意図（「レイヤーを複製」など）で、検証したうえでOp列に展開する関数。**Op**は最小で可逆な状態変更。履歴とジャーナルに残るのはOpと意図ラベル（Intent）だけ。

```
// alto-doc
pub enum Op {
    InsertNode { node: Node, parent: NodeId, index: usize },
    RemoveNode { id: NodeId },                          // 逆操作用に適用時のサブ
ツリーを保存
    MoveNode { id: NodeId, parent: NodeId, index: usize },
    SetProps { id: NodeId, patch: PropPatch },
    SetTiles { layer: NodeId, target: TileTarget, changes: Vec<TileChange> },
    SetSelection { changes: Vec<TileChange> },
    SetVector { id: NodeId, paths: Vec<StyledPath> },
    SetText { id: NodeId, content: TextContent },
    SetAsset { record: AssetRecord },
    SetMeta { patch: MetaPatch },
}
pub enum TileTarget { Content, Mask }
pub struct TileChange { pub key: TileKey, pub before: Option<ContentHash>,
pub after: Option<ContentHash> }
pub struct PropPatch {
    pub name: Option<String>, pub visible: Option<bool>, pub locked:
Option<bool>,
```

```
    pub opacity: Option<f32>, pub blend: Option<BlendMode>, pub transform:
Option<Affine2>,
    pub clip_to_below: Option<bool>, pub effects: Option<Vec<Effect>>,
}

pub struct AppliedOp { pub op: Op, pub inverse: Op }  // 逆操作は適用時に確定させる

/// 編集の「意味」。差分表示や将来の検索・要約に使う
pub struct Intent { pub kind: IntentKind, pub targets: Vec<NodeId>, pub
bounds: Option<Rect> }
pub enum IntentKind {
    BrushStroke { tool: String }, Erase, Transform, AddLayer, RemoveLayer,
Reorder,
    ChangeProps, ApplyEffect, ImportImage, Select, Other(String),
}

pub struct Transaction { pub id: TxId, pub intent: Intent, pub ops:
Vec<AppliedOp>, pub base_revision: u64, pub flush: FlushState }
pub enum FlushState { Complete, PendingReadback(ReadbackTicket) } // GPU読み戻し待ち（第3部 3.6）

pub struct ChangeSet {
    pub nodes: BTreeSet<NodeId>,
    pub structure_changed: bool,
    pub dirty: Vec<(NodeId, Rect)>,      // 文書座標の変更領域
    pub tiles: Vec<(NodeId, TileKey)>,
}

impl Document {
    pub fn begin(&mut self, intent: Intent) -> TxBuilder<'_>;
}
impl TxBuilder<'_> {
    pub fn apply(&mut self, op: Op) -> Result<ChangeSet, DocError>;  // 即時適用（ドラッグ中のプレビューにも使う）
    pub fn commit(self) -> Transaction;
    pub fn rollback(self) -> ChangeSet;                              // 適用済みOpを逆順に戻す
}
```

- **検証してから適用**：Opは適用前にすべて検証し、失敗したOpは状態を一切変えない。Transaction内の途中失敗はロールバックする。
- **ドラッグの畳み込み**：変形のドラッグ中はTransactionを開いたまま`SetProps`を繰り返し適用し、コミット時に同一ノード・同一プロパティを「最初のbefore＋最後のafter」の1つに畳む。

- **不変条件**：任意のOpについて「適用→逆操作の適用」で元の文書と一致する。これをプロパティテストで常時検証する（第6部）。

# 第3部　レンダリング設計

合成は文書座標で行って表示タイルにキャッシュし、ズーム・パン・ビュー回転は最後の提示パスだけで処理する。これでビュー操作はキャッシュを壊さず、再合成が必要なのは編集された領域だけになる。

## 3.1 描画バックエンドの選択

**推奨：Rustの`wgpu`をEngine Worker内で使う。** 比較は下表のとおり。

| 比較項目 | Rust `wgpu`（推奨） | TypeScriptからWebGPUを直接操作 |
|---|---|---|
| WASM⇔JS通信 | 描画判断はRust内で完結。GPU呼び出し自体はwgpuのグルー経由でJSを通るが、タイル合成の呼び出し数は少ない | 何を描くかの判断（シーン・タイル・ダーティ領域）をRustからJSへ毎フレーム渡す必要がある |
| メモリ効率 | タイル画素をWASMメモリから直接`writeTexture`できる | 同じだが、計画データを別途JS側へ複製する |
| Workerとの統合 | Worker内で文書・描画が同居する | 同居できるが、状態がRust/TSの2か所に割れる |
| WebGPUへのアクセス | wgpuが追従するまで新機能を待つことがある | 仕様の新機能を即日使える |
| ネイティブ移植性 | Vulkan・Metal・DX12でそのまま動く | 描画層を書き直しになる |
| 実装の複雑さ | 中。WASMサイズ増とデバッグのしにくさが代償 | 中。境界のデータ設計とJS側の状態管理が代償 |

決め手は「状態を1か所に置ける」ことと「ネイティブ移植」。代償となるWASMサイズとwgpuのWeb対応の成熟度はM0で実測する。問題が出た場合に備え、`alto-gpu`内でwgpuに触る層を薄く保ち、最悪の場合はその層だけをTS実装へ差し替えられるようにする。

## 3.2 座標系と1フレームの処理

- **文書座標**：画素単位。合成はすべてここで行う。
- **表示レベル L**：縮尺 2^-L の表示座標。L = max(0, floor(log2(1 / zoom)))。拡大表示（zoom > 1）はL0を拡大して使う。

- **ビュー変換**：パン・ズーム・回転。提示パスだけが使う。

1. ChangeSetを反映し、`RenderScene`の変更ノードを差し替える。
2. 可視範囲（回転を含む外接矩形）から、必要な表示タイル（L, tx, ty）の集合を求める。
3. キャッシュに無い、または古い表示タイルだけを`RenderPlan`にする。1フレームの作業時間には予算（初期値8ms）を設け、超えた分は次フレームへ回す。未完成のタイルは1段粗いレベルで代用する。
4. コマンドをまとめて記録し、1フレーム1回の`queue.submit`で提出する。
5. 提示：表示タイルをビュー変換付きでキャンバスへ描く。

### 3.3 タイル

| | コンテンツタイル | 表示タイル |
|---|---|---|
| 中身 | レイヤーの正本画素 | 合成結果のキャッシュ |
| 座標 | 文書座標、256px | 表示レベルL、256px |
| GPU形式 | `rgba8unorm`（マスクは`r8unorm`）、テクスチャ配列 | `rgba8unorm` |
| 正本 | CPU側（ContentHash） | なし。いつでも再生成できる |
| 無効化 | 編集されたタイル | 変更領域を祖先方向へ伝播し、全レベルの該当タイル |

- **タイル化は常に使う**：コンテンツは常にタイルで持ち、表示もタイルキャッシュを通す。小さな文書だけ別経路にすると、テストすべき経路が倍になるため。
- **オフスクリーン分離は条件付き**：次のいずれかに当たるノードだけ一時ターゲットへ描き、それ以外は親のターゲットへ直接描く。
  - Isolatedなグループで、不透明度 < 1、合成モード ≠ Normal、マスクかエフェクトのいずれかを持つ
  - 合成モードがハードウェアブレンドで表せない（Normal・Screen・Add以外）
  - クリッピングの基底レイヤー
  - エフェクトを持つノード
- **中間精度**：一時ターゲットは`rgba16float`。8bitのまま重ねるとバンディングが出るため。
- **合成の色空間**：文書の色空間（sRGBのガンマ値）のまま合成する。Photoshopの既定の見た目と合わせるため。リニア合成は文書設定として将来追加する。
- **表示レベルの生成**：MVPでは、ミップマップ付きのコンテンツタイルをレベルLの縮尺で直接描いて合成する。16K四方を超える文書で遅くなったら、レイヤーごとの縮小ピラミッドを追加する（メモリ+33%）。

## 3.4 GPUリソース管理とデータ転送

- **転送は変更分だけ**：毎フレームの転送はなく、変更タイルの`writeTexture`だけ。ブラシはGPU上で直接描き、読み戻しはストローク終了時に1回。
- **読み戻し**：`copyTextureToBuffer`→`mapAsync`。ステージングバッファはリングで再利用する。256px×4byte＝1024byte/行なので、`bytesPerRow`の256byte境界条件を自然に満たす。
- **メモリの目安**：4K（3840×2160）は135タイル。8bitで1タイル256KiB、1レイヤー全面で約34MiB。表示タイルが揃えばパン中はコンテンツタイルが不要なので、GPU予算はコンテンツの作業集合＋表示キャッシュで決まる。
- **一時ターゲット**：（サイズ, 形式）キーのプールで再利用し、フレーム末に返す。

## 3.5 キャッシュ戦略

- 表示タイルは（L, tx, ty）をキーに、描いた時点のシーンのエポックを持つ。変更領域と交差したタイルは「古い」印を付けるが、作り直しが済むまでは古い絵を出し続ける（ちらつき防止）。
- ストローク中のタイルは最優先で作り直す。可視範囲外の古いタイルは後回しにする。
- ビュー回転は提示パスでしか使わないので、回転してもキャッシュは無効にならない。ズームはレベルが切り替わる時（2倍ごと）だけ新レベルを描き、それまでは既存レベルを拡大縮小して見せる。
- 重いグループ（多数のエフェクトなど）の中間結果のキャッシュはM5で検討する。条件は「頻繁に変わらず、再合成コストが閾値を超えるグループ」に限る。

## 3.6 ラスター編集（ブラシ・消しゴム）

1. ストローク開始：Transactionを開き、触れるタイルをGPUへ常駐させる。
2. ストローク中：ダブ列をGPUで**ストロークバッファ**（`r16float`のカバレッジタイル）へスタンプする。フローは重ね塗りで蓄積し、不透明度はバッファの上限として効かせる。画面にはレイヤー＋ストロークバッファを合成したプレビューを出す。
3. ストローク終了：ストロークバッファをレイヤーのタイルへ焼き込む（消しゴムはdestination-out）。
4. 変更タイルだけを読み戻し、ハッシュを計算して`TileStore`へ入れ、`TileChange`（before/after）としてTransactionをコミットする。

- **正本の切り替わり**：焼き込み後のGPUタイルは、読み戻しが終わるまで唯一の最新版になる。この間Transactionは`PendingReadback`。Undoと保存はこのフラッシュ完了を待つ（通常は数フレーム）。
- **GPUタイルへのハッシュ付与**：読み戻し後、GPU上のタイルにも同じハッシュを紐づける。再転送は起きない。

- **履歴は結果で持つ**：ストロークを再実行してUndo/Redoすることはしない。GPUごとに演算結果がわずかに違い、再実行では一致しないため。入力サンプルとブラシ設定はIntentに要約として残す。
- **選択範囲**：文書レベルのR8タイルマップ。ブラシやフィルターはこれをマスクとして参照する（M5）。
- **変形**：レイヤーの`transform`による非破壊変形が基本。画素を確定させる「ラスタライズ」は別のCommandで、新しいタイルを生成する。

### 3.7 非破壊エフェクト

```
// alto-gpu
pub trait EffectKernel {
    fn input_roi(&self, out: IRect) -> IRect;         // 出力に必要な入力範囲（ぼかしなら半径分広がる）
    fn output_bounds(&self, input: IRect) -> IRect;  // 影響が及ぶ範囲（ドロップシャドウなどで広がる）
    fn encode(&self, cx: &mut EffectCx, input: TargetId, out: TargetId, roi: IRect);
}
```

| 種類 | 例 | タイル処理 |
|---|---|---|
| 点処理 | レベル補正、トーンカーブ、色相・彩度 | 周辺を見ないのでタイル単位でそのまま適用 |
| 近傍処理 | ぼかし、シャープ | `input_roi`で広げた入力（タイル＋ハロー）を用意してから適用 |
| 全体参照 | 自動補正、ヒストグラム系 | 先に解析パスを1回走らせ、結果をパラメータとしてキャッシュ |

- **調整レイヤー**は「同じグループ内で自分より下にあるもの全部」が入力。その位置でグループを分離して合成する。
- **Effect Graph**：MVP〜M5は`Vec<Effect>`の直列スタックで足りる。複数入力のエフェクト（置き換えマップなど）が必要になった時点で`EffectGraph { nodes, edges }`へ拡張する。直列スタックはグラフの特殊形なので、データ移行は機械的にできる。

### 3.8 Vector

| 外部ライブラリに任せる | Altoが持つ |
|---|---|
| 曲線の数学とパス表現（`kurbo`） | パスデータのスキーマ、直列化、編集Op |

| 外部ライブラリに任せる | Altoが持つ |
| --- | --- |
| 塗り・線のテッセレーション（`lyon`） | メッシュのキャッシュ、タイルへの描画、アンチエイリアス方式 |
| — | ベクターマスク・クリッピングパスの合成への組み込み |

- 描画はテッセレーション→4×MSAAで表示タイルへ。テッセレーションの許容誤差は表示レベルに合わせ、メッシュはレベル帯ごとにキャッシュする。
- ベクターマスクとクリッピングパスは、パスのカバレッジを`r8unorm`へ描いてから掛け合わせる。
- 代替案はGPUのコンピュートで描く`vello`。品質は高いが重く、Webでの性能が読みにくい。`VectorRasterizer` traitで差し替え可能にしておき、M6で比較する。

### 3.9 Text

| 外部ライブラリに任せる | Altoが持つ |
| --- | --- |
| シェーピング（`rustybuzz`：HarfBuzzのRust移植） | 行分割・段落組版・縦書き・約物・縦中横 |
| フォント解析とアウトライン（`skrifa`） | フォントのアセット管理とフォールバック順 |
| 双方向テキスト・改行規則（`unicode-bidi`・`unicode-linebreak`） | テキスト選択・カーソル位置の計算 |

- 既存の組版ライブラリは日本語縦組みが弱いので、組版は自前にする。データモデルには最初から`WritingMode`を入れておく。
- フォントは文書に埋め込むアセットとして扱う。ブラウザのシステムフォント取得（Local Font Access API）はChromium限定かつ許可制なので前提にしない。
- グリフは大きい文字はベクター経路で、小さい文字はサイズ帯ごとのグリフアトラスで描く。
- 埋め込みの可否はフォントのライセンス（OS/2 `fsType`）で確認し、禁止されているものは警告する。

## 第4部　保存・履歴設計

保存・Undo・版管理は、すべて「ハッシュで名付けた不変オブジェクト＋最後に書く小さな参照（manifest）」という1つの仕組みで作る。オブジェクトは一度書いたら変わらないので、保存の途中で落ちても既存のデータは壊れない。

## 4.1 ファイル形式

論理構造（Alto Package）：

```
manifest.json                形式版・最新スナップショットのハッシュ・作成元
snapshots/<hash>.json        文書ツリー（正規化JSON）
objects/<hh>/<hash>          タイル・アセット・フォント（内容アドレス、圧縮済み）
commits/<hash>.json          版の記録（任意。4.6）
```

物理的には2つの表現を持つ。

| 表現 | 実体 | 向いている用途 |
|---|---|---|
| 単一ファイル `.alto` | ZIP（無圧縮格納。各オブジェクトは自前で圧縮済み） | 受け渡し・通常の保存。既存ツールで中身を確認でき、中央ディレクトリから必要な部分だけ読める |
| フォルダ `.altoproj/` | 同じ構造をそのままディレクトリに展開 | 大きな作品の差分保存、Git管理 |

- **オブジェクトの符号化**：ヘッダ（マジック・コーデック・画素形式）＋本体。圧縮はMVPでは `lz4_flex`（純Rustで高速）。ハッシュは**非圧縮の画素**から計算するので、コーデックを変えても同一性は変わらない。
- **正規化JSON**：キーを整列し、数値の書式を固定し、ノードを木の順に並べる。同じ文書は常に同じバイト列になり、スナップショットのハッシュとGitの差分が安定する。
- **大きなレイヤー**：タイル表はスナップショットJSONに直接書く（4Kで1レイヤー約10KB）。1レイヤーが1024タイルを超えたら、タイル表を独立オブジェクトに分けて参照する。

```
{
  "format": 1,
  "width": 1920, "height": 1080, "color": "srgb", "depth": "u8",
  "root": "0190f3a2-...",
  "nodes": [
    { "id": "0190f3a2-...", "kind": "group", "children": ["0190f3b7-..."] },
    { "id": "0190f3b7-...", "kind": "raster", "name": "背景",
      "opacity": 1.0, "blend": "normal", "transform": [1, 0, 0, 1, 0, 0],
      "tiles": { "0,0": "b3a1...", "1,0": "9c2e..." } }
  ],
  "assets": []
}
```

## 4.2 3層の分離

| 層 | 単位 | 寿命 | 置き場所 |
|---|---|---|---|
| Operation History | Transaction（Op列＋Intent） | セッション中。ジャーナルで異常終了を越える | メモリ＋OPFSジャーナル |
| Document Snapshot | 保存時・コミット時の文書全体（論理的には完全、物理的には変更分だけ増える） | 永続 | パッケージの `snapshots/` |
| Version（Commit） | スナップショット＋親＋メッセージ＋Intentの要約 | 永続 | パッケージの `commits/` |
| Git Adapter | Commit → gitコミット | 外部 | フォルダ形式（M8） |

下の層は上の層を知らない。Undoはスナップショットを使わず、版管理はOpを使わない。Gitはエンジンに一切リンクしない。

## 4.3 Undo / Redo

- Transactionの `inverse` を逆順に適用する。ラスターのUndoは `TileChange` のbeforeとafterを入れ替えるだけで、画素のコピーは起きない。
- Undoスタックが参照するタイルは、ガベージコレクションの根として保持する。
- 上限は「段数（既定100）」と「Undoスタックだけが保持しているタイルの総量（既定512MiB）」の両方。超えたら古い方から捨てる。内容アドレスなので、メモリ上の古いタイルはOPFSへ安全に退避できる。

## 4.4 ラスター差分の方式比較

| 方式 | Undo速度 | 容量 | 実装 | 評価 |
|---|---|---|---|---|
| 内容アドレス化タイル（採用） | 速い（ハッシュの差し替え） | 変更タイル分だけ | 中（GCが必要） | 差分とスナップショットが同じ仕組みになる |
| 変更タイルの画素を履歴に保持 | 速い | 小さい | 中 | 履歴とスナップショットで画素を二重管理することになる |
| 毎回レイヤー全体を保存 | 速い | 非常に大きい | 簡単 | 4Kで1操作34MiB。不採用 |
| 操作の再実行 | 遅い | 最小 | 難しい | GPU間の演算差で結果が一致しない。不採用 |

## 4.5 保存の整合性と復旧

保存の手順:

1. フラッシュ待ち：保留中のGPU読み戻しをすべて完了させる。
2. スナップショットJSONを生成し、ハッシュを確定する。
3. 保存先にまだ無いオブジェクトだけを書く。内容アドレスなので冪等で、途中で失敗しても既存を壊さない。
4. 最後に`manifest.json`を書く。これがコミット点になる。
5. manifestを読み返して検証する。

- **単一ファイル**：File System Access APIの`createWritable()`は一時ファイルに書き、`close()`で初めて置き換える。`abort()`や異常終了では旧ファイルが残るので、この性質をコミット点として使う。ただし毎回全体を書き直すため、大きな作品にはフォルダ形式を勧める。
- **File System Access APIのない環境**（Chromium以外）：保存はダウンロード、開くのはファイル選択で行い、作業中のコピーはOPFSに置く。
- **読み込み時の検証**：オブジェクトはすべて読み込み時にハッシュを照合する。壊れたタイルは欠損として赤い格子で表示して報告し、文書全体は開く。manifestが壊れていれば、有効な最新スナップショットを探して開く。
- **ジャーナル**：開いている文書ごとに、コミット済みTransaction（Op列＋新規タイル）をOPFSへ追記する。書き込みはStorage Worker内の`FileSystemSyncAccessHandle`で行い、各レコードに長さとチェックサムを付けて途中切れを検出する。保存が成功したら切り詰める。
- **異常終了からの復旧**：再起動時、ジャーナルの基点が保存済みスナップショットと一致し、未保存のレコードがあれば復元を提案し、Opを再適用する。

- **ブラウザのキャッシュが消えた場合**：失われるのは最近使ったファイルの一覧とキャッシュだけ。正式な保存ファイルからは完全に開ける。
- **IndexedDBの用途**：最近のプロジェクト（`FileSystemHandle`はIndexedDBに保存できる）、AssetIdとハッシュと所在の索引、設定。GPUの実行時ハンドルは保存しない。

## 4.6 Storage Abstraction

Rust側はバイト列とハッシュだけを扱い、実際のIOはTypeScriptの`StorageProvider`が行う。保存先は「オブジェクト単位で読み書きできるもの」と「アーカイブを丸ごと読み書きするもの」の2種類に分ける。

```
// alto-store
#[async_trait(?Send)]
pub trait ObjectStore {
    async fn has(&self, h: &ContentHash) -> Result<bool, StoreError>;
    async fn get(&self, h: &ContentHash) -> Result<Bytes, StoreError>;
    async fn put(&self, h: &ContentHash, data: Bytes) -> Result<(),
StoreError>; // 冪等
}
#[async_trait(?Send)]
pub trait RefStore {   // manifest のような可変の参照
    async fn read_ref(&self, name: &str) -> Result<Option<Bytes>,
StoreError>;
    async fn write_ref(&self, name: &str, data: Bytes, expect:
Option<ContentHash>) -> Result<(), StoreError>; // 楽観ロック
}
pub trait PackageSink   { /* ZIP ストリームを書き出す */ }
pub trait PackageSource { /* 中央ディレクトリから部分的に読む */ }
```

```
// packages/alto-sdk
export interface ObjectStorageProvider {
  readonly kind: string;   // "fs-access-dir" | "opfs" | "memory" | 将来
"gdrive"
  getObject(hash: string): Promise<Uint8Array | null>;
  putObjects(entries: { hash: string; data: Uint8Array }[]): Promise<void>;
  readRef(name: string): Promise<Uint8Array | null>;
  writeRef(name: string, data: Uint8Array, expectHash?: string):
Promise<void>;
}
export interface ArchiveStorageProvider {
  readonly kind: string;   // "fs-access-file" | "download"
  readArchive(): Promise<Blob>;
  writeArchive(stream: ReadableStream<Uint8Array>): Promise<void>; // 原子的な
置き換えを実装側が保証
}
```

Google Driveなどの外部ストレージは、どちらかのインターフェースを実装するだけで追加できる。エンジン側の変更は不要。

## 4.7 版管理とGitとの接続

```
// alto-doc
pub struct Commit {
    pub parents: Vec<ContentHash>,
    pub snapshot: ContentHash,
    pub message: String,
    pub intents: Vec<IntentSummary>,   // 「背景にブラシ3回」など
    pub created_at: i64,
}

pub enum DiffItem {
    Added(NodeId), Removed(NodeId),
    Moved { id: NodeId, from: (NodeId, usize), to: (NodeId, usize) },
    PropsChanged { id: NodeId, fields: Vec<&'static str> },
    PixelsChanged { id: NodeId, tiles: usize, bounds: IRect },
    AssetChanged(AssetId),
}
pub fn diff(a: &Document, b: &Document) -> Vec<DiffItem>;
```

- **意味的差分**：NodeIdで2つの木を突き合わせる。画素の変化はタイルハッシュの比較だけで求まるので、画素を読まずに「どのレイヤーのどの範囲が変わったか」が分かる。Intentの要約と組み合わせれば、人が読める変更履歴になる。

- **Git Adapter**：Altoのコミットをgitのコミットに写す。フォルダ形式では、スナップショットJSONはテキスト差分になり、オブジェクトは追加されるだけで書き換わらないので、gitと相性がよい。実装はエンジン外の別パッケージ（TSならisomorphic-git、ネイティブならgitoxideが候補）。
- **ガベージコレクション**：コミット・スナップショット・Undoスタック・ジャーナルのいずれからも到達できないオブジェクトを回収する。対象はOPFSの作業領域だけで、ユーザーの保存先からは削除しない（単一ファイルの全体書き直し時に自然に除かれる）。

# 第5部　SDK設計

SDKの約束は3つ。読み取りは同期（Main側ミラーから）、変更は1操作1往復（トランザクション単位）、高頻度の入力とビュー操作は待たない。これでUIはWorkerの応答を待たずに描け、通信回数も操作数に比例しない。

## 5.1 ブラウザランタイム

| スレッド | 中身 | 導入 |
|---|---|---|
| Main | UI、入力の集約、SDKプロキシ、文書ミラー | M0 |
| Engine Worker | WASM（文書・履歴・描画・資源）、GPUDevice、OffscreenCanvas | M0 |
| Storage Worker | OPFSジャーナル（同期アクセスハンドル）、圧縮、ハッシュ計算 | M3 |
| Compute Worker群 | CPUフィルター・符号化の並列処理（WASMスレッド） | M5以降、任意 |

- **SharedArrayBuffer**：`crossOriginIsolated` が真のとき（COOP: `same-origin` ＋ COEP: `require-corp` または `credentialless`）だけ使う。使える場合は入力をリングバッファで渡してpostMessageの遅延を省き、WASMスレッドでCPU処理を並列化する。
- **SABなしが基本線**：外部サイトへの埋め込みなど、COOP/COEPを付けられない環境でも全機能が動くこと。差が出るのは速度だけにする。
- **WASMスレッド**：アトミクス有効版とそうでない版の2つのWASMを用意し、起動時に選ぶ。アトミクス版のビルドには現状nightlyと `build-std` が必要なので、M0で現状を再確認する。
- **WASMメモリ**：wasm32の線形メモリは最大4GiBで、伸ばしたら縮まない。タイル本体はWASMメモリに溜め込まず、作業集合だけを置き、残りはOPFSへ退避する。CPU側タイルキャッシュには予算（既定1GiB）を設ける。
- **非同期タスク**：Worker内は単一スレッドの非同期（`wasm-bindgen-futures`）。GPU読み戻しや保存のIO待ちの間もフレームは回り続ける。

## 5.2 TypeScript API

```
import { AltoEngine } from "@alto/engine";

const alto = await AltoEngine.create({
  canvas,                                    // 内部で
transferControlToOffscreen()
  gpu: { powerPreference: "high-performance", memoryBudgetMB: 512 },
});

const doc = await alto.createDocument({ width: 1920, height: 1080 });

// 読み取りは同期（ミラーから）
const layers = doc.view.root.children;      // NodeView[]
doc.view.on("change", (patch) => renderLayerPanel(patch));

// 変更はトランザクション単位で1往復
const asset = await alto.assets.importImage(file);   // File | Blob →
AssetRef
const { nodeId } = await doc.transact("画像を配置", (tx) => {
  const id = tx.addRasterFromAsset(asset, { parent: doc.view.root.id });
  tx.setTransform(id, { x: 100, y: 200, scaleX: 1, scaleY: 1, rotation: 0 });
  return { nodeId: id };                     // IDはクライアント側で生成するのでそ
の場で使える
});

await doc.undo();
```

`transact`のコールバックはMain側で実行され、Opを記録するだけ。終わった時点でまとめて1回送る。コールバック内でエンジンの計算結果は読めないが、その代わり往復は1回で済む。

```
// 対話的なツール：入力は待たない、終了だけ待つ
const brush = doc.tools.begin("brush", {
  layer: nodeId, tip: { size: 24, hardness: 0.8 }, color: [0, 0, 0, 1],
});
canvas.addEventListener("pointermove", (e) => brush.input(e)); // 内部で集約し、フレーム単位で転送
canvas.addEventListener("pointerup", () => brush.end());        // Promise<TxId>

// ビュー：待たない。座標変換はMain側の行列ミラーで同期計算
alto.viewport.set({ center: [960, 540], zoom: 1.5, rotation: 0 });
const p = alto.viewport.screenToDocument(x, y);

// 保存と読み込み
const target = await alto.storage.pickSaveFile({ suggestedName: "work.alto" });
await doc.save(target);                       // フラッシュ待ちを含む
const reopened = await alto.openDocument(target);
```

イベント：`change`（文書パッチ）、`dirty`（未保存状態の変化）、`saveState`、`gpuLost`/`gpuRestored`、`recoveryAvailable`、`error`。

## 5.3 Rust公開API

```
// alto-engine：ネイティブでもWASMでも同じAPI
pub struct Engine { /* documents, histories, tile store, renderer */ }

impl Engine {
    pub async fn new(gpu: GpuContext, config: EngineConfig) -> Result<Self,
EngineError>;
    pub fn create_document(&mut self, meta: DocumentMeta) -> DocHandle;
    pub fn document(&self, d: DocHandle) -> &Document;
    pub fn transact(&mut self, d: DocHandle, intent: Intent, ops: Vec<Op>) ->
Result<TxResult, EngineError>;
    pub async fn undo(&mut self, d: DocHandle) -> Result<Option<ChangeSet>,
EngineError>; // フラッシュ待ちがあるので async
    pub async fn redo(&mut self, d: DocHandle) -> Result<Option<ChangeSet>,
EngineError>;
    pub fn begin_tool(&mut self, d: DocHandle, spec: ToolSpec) ->
Result<ToolSession, EngineError>;
    pub fn tool_input(&mut self, s: ToolSession, samples: &[StrokeSample]);
    pub async fn end_tool(&mut self, s: ToolSession) -> Result<TxId,
EngineError>;
    pub fn set_viewport(&mut self, v: Viewport);
    pub fn render_frame(&mut self, budget: Duration) -> FrameReport;
    pub async fn save(&mut self, d: DocHandle, sink: &mut dyn PackageSink) ->
Result<SaveReport, EngineError>;
    pub async fn open(&mut self, src: &dyn PackageSource) ->
Result<DocHandle, EngineError>;
}
```

Web上のwgpuの型は`Send`ではないので、`Engine`も`Send`にしない。ネイティブで複数スレッドから使う場合は、呼び出し側が専用スレッドに閉じ込める。

## 5.4 WASM境界とWorker間通信

```
// Main → Worker（返事あり）
type Request =
  | { t: "hello"; protocol: 1; sdk: string }
  | { t: "createDocument"; id: number; meta: DocumentMeta }
  | { t: "transact"; id: number; doc: string; intent: Intent; ops: Op[];
baseRevision: number }
  | { t: "undo" | "redo"; id: number; doc: string }
  | { t: "save" | "open"; id: number; doc?: string; target: StorageTarget };

// Main → Worker（返事なし）
type Stream =
  | { t: "input"; session: number; samples: Float32Array }   // transfer
```

```
  | { t: "viewport"; center: [number, number]; zoom: number; rotation: number
}
  | { t: "tick"; time: number };

// Worker → Main
type Event =
  | { t: "reply"; id: number; ok: true; value: unknown }
  | { t: "reply"; id: number; ok: false; error: AltoErrorData }
  | { t: "patch"; doc: string; revision: number; changes: NodePatch[] }
  | { t: "gpu"; state: "lost" | "restored" };
```

- **制御メッセージ**は普通のオブジェクトを構造化複製で送る。デバッグしやすく、Opは小さいので十分速い。Worker内では`serde-wasm-bindgen`でRustの`Op`へ変換する。
- **大きなデータ**（入力サンプル、画像の画素）は`Float32Array`/`Uint8Array`をtransferする。画素は構造化複製も`serde`も通さず、WASMメモリへ1回コピーするだけ。
- **整合性**：書き込みはWorkerが受け取った順に1本で処理するので競合は起きない。`baseRevision`は、ミラーが古く、削除済みノードを参照したOpを`CONFLICT`として返すために使う。

## 5.5 エラー処理

| コード | 意味 | 回復 |
|---|---|---|
| `INVALID_ARGUMENT` | 引数やOpが不正 | 呼び出し側を直す。状態は変わらない |
| `NOT_FOUND` | NodeId・AssetIdが存在しない | ミラーを更新して再試行 |
| `CONFLICT` | ミラーのrevisionが古い | 最新パッチの適用後に再実行 |
| `GPU_LOST` | デバイス喪失 | 自動復旧。進行中のストロークだけ取り消し |
| `GPU_UNSUPPORTED` | WebGPU非対応・必要機能の不足 | 起動失敗として報告 |
| `STORAGE` | 権限なし・容量不足・書き込み失敗 | 正式ファイルは未変更。再試行か別の保存先 |
| `CORRUPT_DATA` | ハッシュ不一致・形式不正 | 該当部分を欠損扱いにして開く |
| `INTERNAL` | エンジンのバグ（panic） | Workerを再起動し、ジャーナルから復元 |

WASMのpanicはインスタンスを使えなくするので、panic hookで`INTERNAL`を報告し、SDKがWorkerを作り直してジャーナルから復元する。Op適用の経路では`unwrap`を禁止し、失敗は必ず`Result`で返す。

### 5.6 APIのバージョニング

3つの版を独立に管理する。

| 対象 | 方式 | 破壊的変更の扱い |
|---|---|---|
| SDK（`@alto/engine`） | SemVer | メジャー更新のみ。`alto.unstable.*`は対象外で、いつでも変わる |
| プロトコル（SDK⇔WASM） | 整数。`hello`で照合 | 不一致なら起動を拒否し、明確なエラーを出す（古いWASMのキャッシュ対策） |
| ファイル形式 | manifestの整数 | 読み込み時にRustのマイグレーションで最新へ変換し、書き込みは常に最新。未知のノードは`Unsupported`として保持し、保存時にそのまま書き戻す |

## 第6部　開発計画

MVPはM4で完成し、その時点で「Rust製エンジンがWASMとしてブラウザで動き、WebGPUで描画し、編集状態を保存・復元できる」が成立する。最初のM0は、設計の前提（Worker内のwgpu、読み戻し、WASMサイズ）を実機で確かめるための捨てるつもりの検証にする。

### 6.1 MVPの定義

- [ ] Chrome/EdgeのWebGPUで、Engine WorkerがOffscreenCanvasへ描画する
- [ ] 文書の作成、PNG/JPEGの読み込み（ラスターレイヤー化）
- [ ] レイヤーの追加・削除・並べ替え・グループ・表示切替・不透明度、合成モードはNormal・Multiply・Screen
- [ ] 移動・拡大縮小・回転（非破壊変形）
- [ ] ズーム・パン・ビュー回転
- [ ] 丸ブラシと消しゴム（筆圧対応、GPU描画→読み戻しで確定）
- [ ] ラスターを含むUndo/Redo
- [ ] `.alto`の保存・読み込み、ジャーナルからの異常終了復旧
- [ ] ベンチマーク B1〜B7 の目標を満たす

MVPに入れないもの：選択範囲、マスク、調整レイヤー、エフェクト、ベクター、テキスト、版管理。データモデルの枠だけはM1で用意する。

## 6.2 リポジトリ構成

```
alto/
├── Cargo.toml                 Rust workspace
├── crates/
│   ├── alto-core/
│   ├── alto-raster/
│   ├── alto-doc/              History を含む
│   ├── alto-store/
│   ├── alto-gpu/
│   │   └── shaders/           WGSL（include_str! で取り込む）
│   ├── alto-vector/           M6 で追加
│   ├── alto-text/             M7 で追加
│   ├── alto-engine/
│   └── alto-wasm/
├── packages/
│   └── alto-sdk/              @alto/engine（Worker エントリを同梱）
├── examples/
│   ├── browser-playground/
│   └── native-viewer/         ネイティブでの動作確認
├── tests/
│   ├── golden/                参照画像
│   └── e2e/                   Playwright
└── benches/
```

シェーダーはトップレベルではなく`alto-gpu`の中に置く。シェーダーとそれを呼ぶRustコードは一緒に変わるため。

# 6.3 マイルストーン




マイルストーン·M0〜M8、M4がMVP

M0〜M4は前の段階の完了を前提に順に進める。M5〜M8は互いの依存が薄いので、必要に応じて順番を入れ替えてよい。

## 6.4 技術的リスク

| リスク | 影響 | 対策 | 確認時期 |
|---|---|---|---|
| WebGPUのWorker対応とブラウザ差 | Firefox・Safariで起動しない | M0で実機確認。MVPはChrome/Edgeを正式対応とし、他は順次 | M0 |
| wgpuのWebバックエンドの成熟度・WASMサイズ | 初回ロードが重い、不具合 | サイズ計測と`wasm-opt`。wgpuに触る層を薄く保ち、差し替え可能にする | M0 |
| GPU読み戻しの遅延 | ストローク確定・保存が遅れる | 変更タイルだけ読み戻す。待つのはUndoと保存の時だけ | M0・M4 |
| wasm32のメモリ上限（4GiB、縮小不可） | 大きな文書で落ちる | タイルの退避と予算管理、空タイルを持たない | M2 |
| GPUデバイス喪失 | 描画が止まる | GPUは常にキャッシュ扱い。`device.destroy()`で故意に起こして試験する | M2 |
| GPU間の演算差 | 画像テストが不安定 | 許容誤差付きの比較。正本はCPU側のハッシュ | M2 |
| File System Access APIがChromium限定 | 他ブラウザで上書き保存できない | ダウンロード保存＋OPFSの作業コピー | M3 |
| COOP/COEPの要件 | 埋め込み先と両立しない | SABなしを基本線、スレッド版は任意 | M0 |
| 日本語縦組みの複雑さ | テキストが遅れる | 自前組版を段階導入し、M7を独立させる | M7 |
| フォント埋め込みのライセンス | 配布時の問題 | `fsType`を確認して警告 | M7 |

## 6.5 テスト戦略

- **単体＋プロパティテスト**（`proptest`）：Op適用→逆適用で元通り、直列化の往復でバイト一致、ID衝突なし。
- **描画のゴールデンテスト**：ネイティブwgpu（CIではソフトウェアVulkan）で描き、参照画像と許容誤差付きで比較する。合成モード×不透明度×マスクの組み合わせ。
- **CPU/GPU一致テスト**：同じダブ列をCPU描画器とGPUで描き、差が閾値以内か確かめる。

- **ブラウザE2E**：PlaywrightでChromeを起動し、SDK経由の操作→スクリーンショット比較。
- **障害注入**：保存中のWorker強制終了、書き込み失敗、デバイス喪失、ハッシュ不一致データの読み込み。
- **ファジング**（`cargo-fuzz`）：パッケージの読み込み、JSONのマイグレーション。

## 6.6 ベンチマーク設計

基準機（GPU・ブラウザ・OS）をM0で決めて固定する。時間はWorker内の`performance.now()`で測り、`timestamp-query`が使える環境ではGPU時間も記録する。目標値は初期案で、M0の実測で見直す。

| ID | シナリオ | 指標 | MVP目標 |
|---|---|---|---|
| B1 | 4K・20ラスターレイヤーで連続パン | フレーム時間 p95 | 16.7ms以下 |
| B2 | 同じ文書でズーム段階を変更 | 新レベルが全て描き終わるまで | 250ms以下 |
| B3 | 4Kレイヤーへブラシ（半径50px）を連続描画 | 入力から表示までの遅延 | 2フレーム以内 |
| B4 | 4K全面ストロークのUndo | 完了まで | 50ms以下 |
| B5 | 4K・20レイヤーの保存／読み込み | 所要時間 | 3秒以下／2秒以下 |
| B6 | 同じ文書の常駐メモリ | GPU／WASM | 512MiB以下／1GiB以下 |
| B7 | 起動（WASMキャッシュ済み） | 初回描画まで | 1.5秒以下 |

## 6.7 最初の作業（M0）

1. `alto-wasm`の最小構成で、Worker内にwgpuのデバイスとOffscreenCanvasのSurfaceを作る。
2. 256×256のテクスチャ配列にタイルを100枚上げ、パン・ズームしながら描く。
3. 1タイルを書き換えて読み戻し、往復時間を測る。
4. Chrome・Firefox・Safariで動作を記録し、WASMのサイズ（圧縮後）を測る。
5. 結果をもとに、この設計書の3.1・5.1・6.4を更新する。